FIRST REVIEW REPORT

Cloud Computing

Predictive Auto Scaling Using Workload Prediction, Hysteresis and Safety-Aware Capacity Planning

Domain: Cloud Computing, Distributed Systems, Machine Learning, Site Reliability Engineering Technology Stack: Python, Pandas, NumPy, Matplotlib, Flask, Docker, Locust, Google Cloud Platform

Table of Contents

1. Abstract

Session: 2026-27

Mentor: Dr. Manoj Kumar Patra

2. Introduction and Problem Statement

2.1 Background

2.2 Auto Scaling Challenges

2.3 Reactive Autoscaling Trap

2.4 Gap in Existing Solutions

2.5 Problem Statement

2.6 Objectives

3. Literature Review

3.1 Synthesis

4. Proposed Architecture and Methodology

4.1 Core Idea

4.2 Execution Flow

4.3 System Architecture

4.4 Decision Logic

4.5 Implementation Architecture

4.6 Interactive Demonstrator

4.7 Design Rationale

5. Experimental Setup and Benchmarking

5.1 Local Environment

5.2 Workload Generation

5.3 Experimental Design

5.4 Cases Solved

5.5 Performance Metrics

6. Results and Analysis

7. Scope and Limitations

8. Future Scope

9. Conclusion

References

1. Abstract

Cloud applications experience continuously changing workloads. Traditional threshold-based auto scaling reacts to the current state of the system, but scaling actions may be delayed and fluctuating demand can cause repeated scale-out and scale-in operations. This project develops a predictive auto scaling controller that estimates near-future workload and combines the prediction with capacity planning, a safety margin, hysteresis, consecutive-observation rules, and cooldown control. The baseline controller uses current utilization thresholds, while the proposed controller uses a lightweight trend-adjusted moving-average predictor. Both controllers are evaluated with identical synthetic workload traces containing stable periods, gradual increases, spikes, bursts, low-demand periods, and stress conditions. The prototype measures instance count, scaling actions, response time, SLA violations, and estimated compute-resource usage. The project is implemented locally in Python and is designed for optional mapping to cloud services such as Google Cloud Managed Instance Groups, Load Balancing, and Cloud Monitoring. The numerical results in this review are controlled simulation results rather than production-cloud measurements.

2. Introduction and Problem Statement

2.1 Background

Cloud computing provides elasticity: application capacity can be increased or decreased according to demand. Auto scaling is the mechanism that performs this capacity adjustment. Scale-out adds computing instances when demand increases, while scale-in removes instances when demand decreases. For a web service, this is useful because the number of users and requests can vary significantly over time.

The central difficulty is that workload changes and resource provisioning do not occur at exactly the same time. A new instance requires initialization, application startup, and readiness before it can contribute useful capacity. Therefore, a controller that waits until overload is already visible can react too late.

2.2 Auto Scaling Challenges

Scaling delay: capacity becomes available only after a scale-out operation completes.

Oscillation: a workload near a threshold can cause repeated scale-out and scale-in decisions.

Single-metric decisions: CPU alone may not represent request pressure or latency.

Over-provisioning: conservative rules can keep unnecessary instances running.

Under-provisioning: sudden spikes can exceed currently available capacity.

Cost-performance trade-off: protecting latency by keeping many instances active increases resource consumption.

2.3 Reactive Autoscaling Trap

A reactive controller can create an undesirable feedback pattern. A sudden spike raises utilization, the controller starts new instances, and the system temporarily continues to operate under pressure. If the controller immediately reacts again to the resulting utilization, capacity can change repeatedly. This project uses the term reactive autoscaling trap for this feedback behavior in the context of the controlled simulation.

Figure 1: Reactive autoscaling feedback trap considered by the project.

2.4 Gap in Existing Solutions

Native cloud autoscalers are useful and robust, but a mini-project needs an experimental framework in which different decision policies can be replayed against exactly the same workload. The gap addressed here is therefore not the

absence of cloud autoscaling, but the need for an understandable predictive controller and a controlled benchmark that exposes the trade-off among responsiveness, stability, SLA behavior, and resource usage.

The proposed approach also separates forecasting from the final scaling decision. This is important because a forecast can be wrong; current-load safety checks, hysteresis, and cooldown remain active as protective mechanisms.

2.5 Problem Statement

The problem is to design, implement, and evaluate a lightweight predictive horizontal auto scaling mechanism that anticipates near-future workload while limiting unnecessary capacity changes. Given a request-rate time series, per-instance capacity, and minimum/maximum instance limits, the system should estimate required capacity, decide when to scale out or scale in, and maintain acceptable response-time/SLA behavior under changing workloads.

2.6 Objectives

Develop a reactive threshold-based baseline for comparison.

Generate controlled workload patterns representing normal and abnormal demand.

Predict near-future request demand using a lightweight trend-adjusted moving average.

Calculate required instances using predicted demand and a safety margin.

Use hysteresis and cooldown to reduce scaling oscillation.

Compare baseline and proposed controllers using common performance and resource metrics.

Provide a reproducible local implementation and an optional cloud deployment mapping.

3. Literature Review

The project is positioned at the intersection of reactive autoscaling, predictive resource management, and control stability. Reactive policies use current measurements such as CPU utilization, request rate, queue length, or latency. Predictive policies estimate future demand from historical observations. Hybrid policies combine the two so that prediction provides anticipation while reactive rules provide protection when forecasts are inaccurate.

3.1 Synthesis

The design follows a hybrid principle: prediction estimates what is likely to happen next, while current utilization and control safeguards decide whether a capacity change is safe. The project intentionally uses a lightweight predictor so that the algorithm can be understood, implemented, and tested within a mini-project environment.

4. Proposed Architecture and Methodology

4.1 Core Idea

The proposed controller follows a five-stage pipeline: monitoring, prediction, capacity planning, decision control, and evaluation. The predictor uses a rolling history of workload measurements and a trend adjustment. The capacity planner converts predicted request rate into an integer instance requirement. A 20% safety margin is applied to reduce the risk of under-provisioning.

Required Instances = ceil(Predicted Request Rate × (1 + Safety Margin) / Per-Instance Capacity)

Example: if predicted demand is 900 requests/minute, one instance can process 220 requests/minute, and the safety margin is 20%, then required instances = ceil(900 × 1.20 / 220) = 5.

4.2 Execution Flow

Collect workload and system measurements.

Update the rolling workload history.

Predict the near-future request rate.

Calculate required instances with the safety margin.

Compare required capacity with current capacity.

Apply scale-out or scale-in conditions.

Apply cooldown so repeated actions are not performed immediately.

Record the resulting metrics for benchmarking.

4.3 System Architecture

Figure 2: Proposed predictive auto scaling architecture and methodology.

4.4 Decision Logic

The baseline uses a 70% scale-out threshold and a 30% scale-in threshold with a five-minute cooldown. The proposed controller uses the predicted requirement as the primary capacity signal but also checks current load. Scale-out is allowed when predicted requirement exceeds current capacity or current utilization remains high. Scale-in is more conservative: the predicted requirement must be lower and current utilization must remain low for several observations. Both policies enforce a minimum of one and maximum of eight instances in the default simulation.

Figure 3: Decision flow used by the project.

4.5 Implementation Architecture

Figure 4: Local implementation architecture showing the components used to build and evaluate the prototype.

The local implementation separates workload generation, metric collection, prediction, decision control, instance simulation, result storage, and visualization. This separation allows the same workload to be replayed against different scaling policies.

4.6 Interactive Demonstrator

An interactive browser-based demonstrator is included with the project package. It is a self-contained HTML file and does not require a web server. Users can switch between the reactive baseline and proposed predictive controller, move through the workload timeline, play/pause the simulation, and click architecture components to view their implementation role.

File: interactive/interactive_autoscaling_demo.html

4.7 Design Rationale

Lightweight: the predictor is computationally simple and suitable for a mini-project.

Predictive: capacity planning uses near-future demand instead of only current utilization.

Safe: a 20% margin, hysteresis, and cooldown reduce aggressive decisions.

Reproducible: both policies consume the same workload trace.

Extensible: the predictor can later be replaced by LSTM, GRU, XGBoost, or another model.

5. Experimental Setup and Benchmarking

5.1 Local Environment

5.2 Workload Generation

The default workload contains stable periods, gradual variation, spikes, and bursts over a 360-minute simulation. The same request-rate sequence is used for both controllers. Optional Locust testing can replace the synthetic simulator with generated HTTP traffic when a real service endpoint is available.

Figure 5: Synthetic workload used in the simulation.

5.3 Experimental Design

Figure 6: Controlled benchmarking design.

Experiment A is the reactive baseline. Experiment B is the proposed predictive controller. The workload trace, capacity limits, observation interval, and simulation duration remain fixed; the scaling policy is the main independent variable.

5.4 Cases Solved

5.5 Performance Metrics

6. Results and Analysis

The current simulation results are included below. They should be interpreted as results under the project's defined workload and response-time model, not as measurements of a live cloud deployment.

Figure 7: Workload prediction behavior.

Figure 8: Baseline and proposed instance usage.

Figure 9: SLA comparison.

Figure 10: Cumulative SLA violations; the current default trace produces zero violations for both policies.

7. Scope and Limitations

The workload is synthetic and may not represent all real cloud workloads.

The response-time relationship is simulated rather than measured from a real application.

Instance startup time and cloud scheduling delays are simplified.

Estimated cost is a resource-cost unit, not an actual cloud billing amount.

The default predictor is lightweight and may be less accurate than advanced forecasting models.

The controller focuses on horizontal scaling and does not fully model vertical scaling.

Database bottlenecks, multi-tier dependencies, network congestion, and heterogeneous instances are outside the current scope.

Prediction errors can still produce delayed or unnecessary scaling; safeguards reduce but do not eliminate this risk.

8. Future Scope

Replace the moving-average predictor with LSTM, GRU, XGBoost, or another time-series model.

Add prediction-confidence intervals and adaptive safety margins.

Use multiple signals: request rate, CPU, memory, queue length, latency, and error rate.

Measure actual VM/container startup and shutdown time.

Deploy the controller on GCP, AWS, or Azure and compare it with native autoscaling.

Integrate with Kubernetes HPA/KEDA.

Use real workload traces and longer-duration experiments.

Add cost-aware optimization using actual cloud pricing.

Extend the method to multi-tier microservices.

Evaluate energy and carbon-aware scaling.

Add anomaly detection for unexpected workload spikes.

9. Conclusion

This project presents a reproducible predictive auto scaling framework for studying the relationship between workload, performance, stability, and resource usage. The baseline provides a transparent reactive reference, while the proposed controller combines lightweight forecasting with capacity planning, safety margin, hysteresis, and cooldown. The implementation diagrams and interactive demonstrator make the internal decision process visible rather than treating autoscaling as a black box. The framework can subsequently be connected to real cloud infrastructure and real load-testing traffic for a production-oriented evaluation.

References

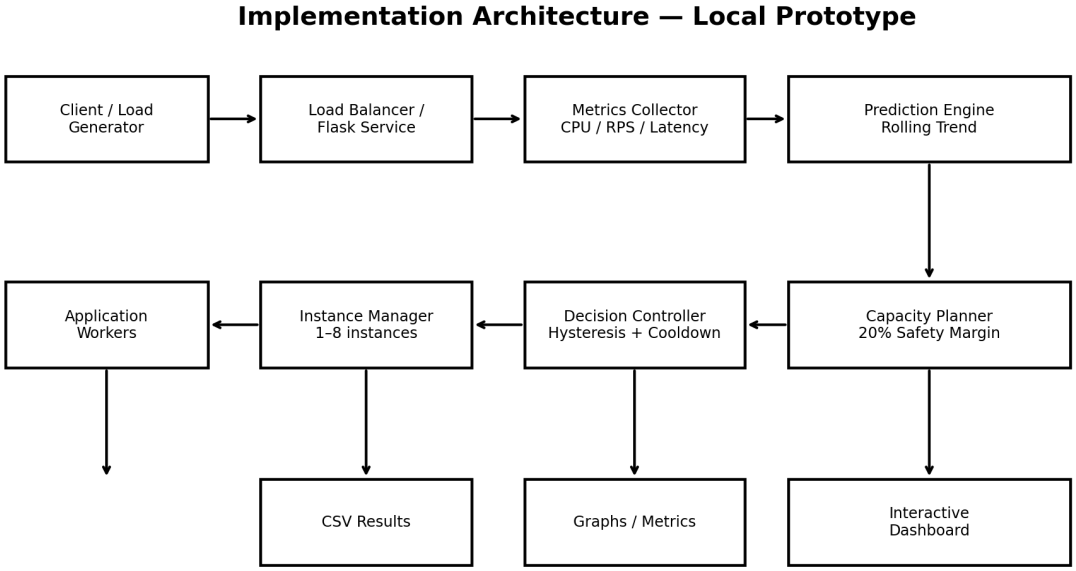
Google Cloud Compute Engine documentation — Autoscaling and Managed Instance Groups.

Locust documentation — HTTP load testing and user behavior simulation.

Scikit-learn documentation — machine learning and evaluation utilities.

Project source code, workload dataset, simulation outputs, and diagrams included in the accompanying project package.

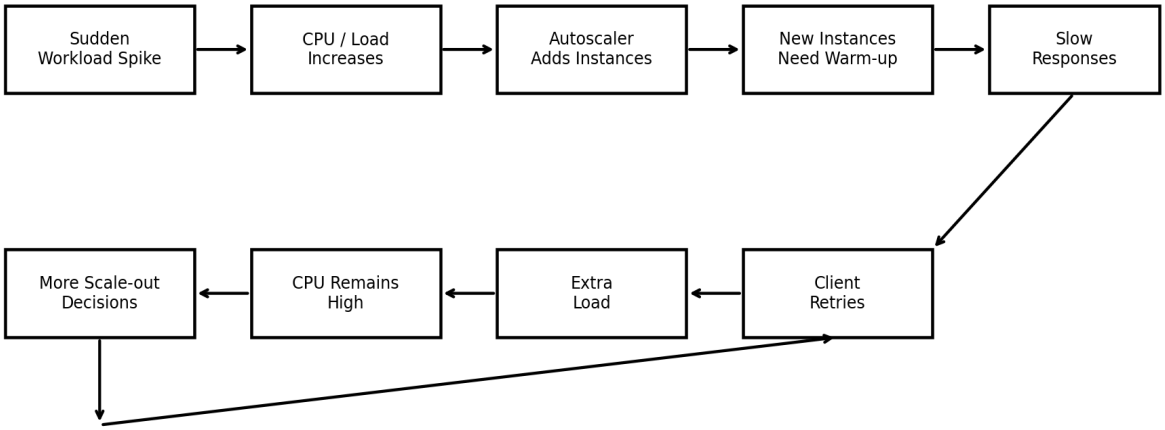
Implementation Architecture



Python + Pandas + NumPy + Matplotlib + Flask + Docker + Locust

Reactive Autoscaling Trap

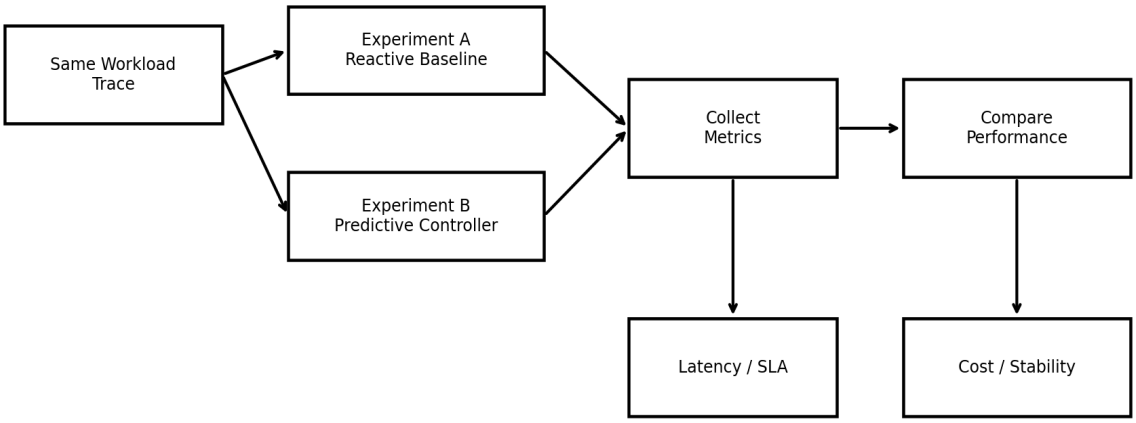
Figure: Reactive Autoscaling Feedback Trap



Result: resource usage can increase while useful throughput fails to recover quickly.

Experimental Design

Figure: Controlled Benchmarking Design



Independent variable: scaling policy. All other workload and capacity parameters remain fixed.